PES-VCS

Building a Version Control System from Scratch

Lab Report

Operating Systems

PES University

Name: Riyana Karthik

SRN: PES1UG24AM226

Section: D

Date: 21/4/2026

Objective

Build a local version control system (PES-VCS) that tracks file changes, stores snapshots efficiently using content-addressable storage, and supports commit history. Every component maps directly to operating system and filesystem concepts covered in the course.

Platform: Ubuntu 22.04

System Overview

PES-VCS implements five core commands across four phases:

- pes init — Create the .pes/ repository structure
- pes add <file>... — Stage files (hash + update index)
- pes status — Show modified/staged/untracked files
- pes commit -m <msg> — Create a commit from staged files
- pes log — Walk and display commit history

The .pes/ directory structure mirrors Git's internals:

```
.pes/
├── objects/          # Content-addressable blob/tree/commit storage
│   ├── 2f/           # Sharded by first 2 hex chars of hash
│   └── a1/
├── refs/
│   └── heads/
│       └── main      # Branch pointer (file containing commit hash)
├── index             # Staging area (text file)
└── HEAD              # Current branch reference
```

Phase 1: Object Storage Foundation

Filesystem Concepts: Content-addressable storage, directory sharding, atomic writes, hashing for integrity

1.1 What Was Implemented

Two functions were implemented in object.c:

object_write

Stores data in the content-addressable object store. Steps:

- Prepends a type header: blob <size>\0, tree <size>\0, or commit <size>\0
- Computes SHA-256 hash of the full object (header + data)
- Writes atomically using the temp-file-then-rename pattern
- Shards files into subdirectories using the first 2 hex characters of the hash

object_read

Retrieves and verifies data from the object store. Steps:

- Reads the object file and parses the header to extract type and size
- Verifies integrity by recomputing the SHA-256 hash and comparing it to the filename
- Returns only the data portion (after the null byte separator)

1.2 Key Concepts

Content-addressable storage provides three properties:

- Deduplication: identical files are stored exactly once (same content = same hash)
- Integrity: the hash acts as a checksum — any corruption is detectable
- Immutability: changing content produces a different hash, creating a different object

Objects are sharded into subdirectories by the first two hex characters of their hash to prevent performance degradation from directories with millions of files.

1.3 Testing

Run the Phase 1 test suite:

```
make test_objects
./test_objects
```

The test verifies:

- Blob storage and retrieval (write → read back → compare)
- Deduplication (same content → same hash → stored once)
- Integrity checking (detects corrupted objects)

1.4 Screenshots

Screenshot 1A — ./test_objects output showing all tests passing:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B — find .pes/objects -type f showing sharded directory structure:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2: Tree Objects

Filesystem Concepts: Directory representation, recursive structures, file modes and permissions

2.1 What Was Implemented

Three functions were implemented in tree.c:

tree_parse

Parses raw binary tree data into a Tree struct. The binary format for each entry is:

```
"<mode> <name>\0<32-byte-hash>"
```

Mode is ASCII octal (e.g., 100644 for regular files), name is a null-terminated string, and hash is 32 raw bytes.

tree_serialize

Serializes a Tree struct back to binary format. Entries must be sorted by name before serialization to ensure deterministic hashing — the same set of files always produces the same tree hash.

tree_from_index

Builds a complete tree hierarchy from the staging index. Handles nested paths (e.g., src/main.c creates a src subtree), writes all tree objects to the object store, and returns the root tree hash. This function is called by pes commit to create the project snapshot.

2.2 File Mode Values

Mode	Meaning
100644	Regular file, not executable
100755	Regular file, executable
040000	Directory (tree)

2.3 Testing

```
make test_tree
./test_tree
```

The test verifies:

- Serialize → parse roundtrip preserves all entries, modes, and hashes
- Deterministic serialization (same entries in any order → identical binary output)

2.4 Screenshots

Screenshot 2A — ./test_tree output showing all tests passing:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B — xxd of a raw tree object (first 20 lines):

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ xxd .pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383
e11934310a10a030f6 | head -20
00000000: 7472 6565 2039 3600 3130 3036 3434 2062  tree 96.100644 b
00000010: 7965 2e74 7874 00e6 7ed6 6bcb 4a70 8250  ye.txt..~.k.Jp.P
00000020: a89f 315d 4d8b b927 0334 3c84 df83 4438  ..1]M..'.4<...D8
00000030: 880e 13a6 8652 bc31 3030 3634 3420 6865  .....R.100644 he
00000040: 6c6c 6f2e 7478 7400 05c8 d777 d0be 8574  llo.txt....w...t
00000050: 8b67 88d1 b5ca f7d6 9754 e6e6 8060 c265  .g.......T...`.e
00000060: ca21 cbd6 c05e 5f43                      .!...^_C
```

Phase 3: The Index (Staging Area)

Filesystem Concepts: File format design, atomic writes, change detection using metadata

3.1 What Was Implemented

Four functions were implemented in index.c:

index_load

Reads the text-based .pes/index file into an Index struct. If the file does not exist, initializes an empty index. Each line has the format:

```
<mode> <hash-hex> <mtime> <size> <path>
```

index_save

Writes the index atomically using the temp-file + rename pattern. Sorts entries by path before writing, and calls fsync() on the temp file before renaming to ensure durability even if the system crashes.

index_add

Stages a file: reads its contents, writes a blob to the object store, and updates the index entry for that path. Uses index_find to check for an existing entry and update it in-place if present.

index_status

Compares the index, working directory, and last commit to print staged changes, unstaged changes, and untracked files. Expected output format:

```
Staged changes:
    new file:   hello.txt
    modified:   src/main.c

Unstaged changes:
    modified:   README.md
    deleted:    old_file.txt

Untracked files:
    notes.txt
```

3.2 Testing

```
make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
```

3.3 Screenshots

Screenshot 3A — pes init → pes add → pes status sequence:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ echo "hello" > file1.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ echo "world" > file2.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes add file1.txt file2.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  commit.h
  untracked:  README.md
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  pes.c
  untracked:  index.c
  untracked:  Makefile
  untracked:  tree.c
  untracked:  object.c
  untracked:  commit.c
  untracked:  tree.h
```

Screenshot 3B — cat .pes/index showing the text-format index:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776757918 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776757921 6 file2.txt
```

Phase 4: Commits and History

Filesystem Concepts: Linked structures on disk, reference files, atomic pointer updates

4.1 What Was Implemented

Three functions were implemented in commit.c, plus cmd_commit in pes.c:

head_read

Reads the commit hash that HEAD currently points to. Follows symbolic references: HEAD → branch file → commit hash. Returns NULL for the initial state (no commits yet).

head_update

Atomically updates the current branch reference file to contain a new commit hash. This is the single atomic operation that publishes a commit — after this write, the new commit becomes reachable.

commit_create

The main commit function. Steps:

- Builds a root tree from the index using tree_from_index()
- Reads the current HEAD hash as the parent commit (empty for first commit)
- Retrieves the author string from the PES_AUTHOR environment variable
- Serializes and writes the commit object
- Updates HEAD via head_update()

cmd_commit (in pes.c)

Parses -m <message> from command-line arguments. On missing -m, prints: error: commit requires a message (-m "message"). On success, prints: Committed: <first-12-hex-chars>... <message>.

4.2 Testing

```
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
make test-integration
```

4.3 Screenshots

Screenshot 4A — pes log output with three commits:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ echo "Hello" > hello.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes add hello.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 9d4d3dc19f4f... Initial commit
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ echo "World" >> hello.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes add hello.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes commit -m "Add world"
Committed: 35881e7a06aa... Add world
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ echo "Goodbye" > bye.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes add bye.txt
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes commit -m "Add farewell"
Committed: ed1d0eee5465... Add farewell
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ ./pes log
commit ed1d0eee54654ffdc12fd23d54f95bd8c88ece4369a0a8893242a1ff401919b4
Author: Your Name <PES1UG24AM226>
Date:   1776758827

    Add farewell

commit 35881e7a06aacb3eb36a966c82c8b4622833e23b693a8ea23035017b602da11c
Author: Your Name <PES1UG24AM226>
Date:   1776758807

    Add worldell

commit 9d4d3dc19f4f98e860c6b81f13ca26a8d67dc6cec1fc860f9f06ad438d86bb24
Author: Your Name <PES1UG24AM226>
Date:   1776758791

    Initial committial comQ
```

Screenshot 4B — find .pes -type f | sort showing object growth:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/35/881e7a06aacb3eb36a966c82c8b4622833e23b693a8ea23035017b602da11c
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/9d/4d3dc19f4f98e860c6b81f13ca26a8d67dc6cec1fc860f9f06ad438d86bb24
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ed/1d0eee54654ffdc12fd23d54f95bd8c88ece4369a0a8893242a1ff401919b4
.pes/refs/heads/main
```

Screenshot 4C — cat .pes/refs/heads/main and cat .pes/HEAD:

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ cat .pes/refs/heads/main
ed1d0eee54654ffdc12fd23d54f95bd8c88ece4369a0a8893242a1ff401919b4
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Screenshot — Full integration test (make test-integration):

```
ubuntu@ubuntu:~/PES1UG24AM226-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 808d0e9ea7b3... Initial commit

Log after first commit:
commit 808d0e9ea7b3f7ed7efc84e437beae0d4fce638b41eff54f46b8b47bfe456a80
Author: Your Name <PES1UG24AM226>
Date:   1776758877

    Initial commit
```

```
--- Second Commit ---
Committed: 81d9016b0348... Update file.txt

--- Third Commit ---
Committed: 8da8feec5a04... Add farewell

--- Full History ---
commit 8da8feec5a049c3f2e93390efd3790b453bbb3d6ab741be8bc72f5aa2f2492bf
Author: Your Name <PES1UG24AM226>
Date:   1776758877

    Add farewell

commit 81d9016b0348c1db54a90c2ef335de8193883c1fc7b52b92a6d61279f46a74ae
Author: Your Name <PES1UG24AM226>
Date:   1776758877

    Update file.txt

commit 808d0e9ea7b3f7ed7efc84e437beae0d4fce638b41eff54f46b8b47bfe456a80
Author: Your Name <PES1UG24AM226>
Date:   1776758877

    Initial committial comQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
8da8feec5a049c3f2e93390efd3790b453bbb3d6ab741be8bc72f5aa2f2492bf
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/80/8d0e9ea7b3f7ed7efc84e437beae0d4fce638b41eff54f46b8b47bfe456a80
.pes/objects/81/d9016b0348c1db54a90c2ef335de8193883c1fc7b52b92a6d61279f46a74ae
.pes/objects/8d/a8feec5a049c3f2e93390efd3790b453bbb3d6ab741be8bc72f5aa2f2492bf
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5: Analysis — Branching and Checkout

Q5.1 — Implementing pes checkout <branch>

Question: *A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?*

Answer

A checkout operation involves two distinct responsibilities: updating the repository metadata and synchronising the working directory.

Files that change in .pes/:

- HEAD is rewritten to contain ref: refs/heads/<branch>. If the branch does not yet exist (pes checkout -b), a new file is created at .pes/refs/heads/<branch> containing the current commit hash.
- If switching to an existing branch, HEAD simply points to that branch's ref file. No object data changes — only the symbolic pointer.

What must happen to the working directory:

- Resolve the target branch's commit hash from its ref file.
- Read the root tree object referenced by that commit.
- Walk the target tree recursively and compare every path to the current HEAD's tree.
- For each file that exists in the target tree but not the current tree: create the file by reading its blob from the object store.
- For each file that exists in the current tree but not the target tree: delete the file from the working directory.
- For each file that exists in both trees but with different hashes: overwrite the working-directory file with the content of the target blob.
- Leave files that are identical in both trees untouched.

What makes checkout complex:

- Dirty working directory detection: Before overwriting any file, the system must check whether the working-directory version differs from the current HEAD version AND from the staged index version. If both differ (uncommitted local changes that conflict with the branch switch), checkout must refuse to avoid silently destroying the user's work.
- Untracked file collision: If a file is untracked in the current state but exists in the target branch's tree, checkout must refuse to overwrite it, since the user never asked PES-VCS to manage that file.
- Partial failure: The working-directory update involves many file operations. If the process is interrupted mid-way (e.g., by a crash), the repository can be left in an inconsistent state. Real implementations handle this with careful ordering and rollback strategies.
- Recursive tree traversal: Nested subdirectories require reading multiple tree objects and creating or deleting entire directory hierarchies, not just individual files.

Q5.2 — Detecting Dirty Working Directory Conflicts

Question: *When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this conflict using only the index and the object store.*

Answer

A conflict exists when three conditions are simultaneously true for a given file path:

- The file exists in the current branch's committed tree (it is a tracked file).
- The working-directory version differs from the index version (the user has made unstaged changes), OR the index version differs from the last committed version (staged but uncommitted changes exist).
- The target branch's committed tree has a different blob hash for that path than the current branch's committed tree (the branches disagree about this file).

Detection algorithm using only the index and the object store:

- Step 1 — Identify which files differ between branches: Read the root tree of the current HEAD commit and the root tree of the target branch's HEAD commit. Walk both trees recursively and collect all paths where the blob hashes differ (or where one tree has the path and the other does not).
- Step 2 — For each conflicting path, check whether the working directory is clean: Load the index and look up the path. Compare the index entry's hash to the current HEAD's blob hash for that path. If they differ, the file has staged but uncommitted changes. Additionally, read the actual file from disk, compute its SHA-256, and compare to the index hash. If they differ, the file has unstaged modifications.
- Step 3 — Refuse if any conflicting path is dirty: If any file that differs between the two branches is also dirty (staged or unstaged changes), print an error and abort the checkout. The error should list the specific conflicting files so the user can resolve them.

This approach is efficient because:

- No working-directory reads are needed for files that are identical between branches — they cannot cause conflicts.
- The index stores the mtime and size metadata, enabling a fast pre-check before computing full SHA-256 hashes of working-directory files.
- The object store provides the ground truth for committed state, requiring no traversal of the entire history.

Q5.3 — Detached HEAD

Question: *"Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?*

Answer

In normal operation, HEAD contains a symbolic reference such as ref: refs/heads/main. In detached HEAD state, HEAD contains a raw commit hash directly — for example, a1b2c3d4e5f6.... This

occurs when a user checks out a specific commit rather than a branch name (e.g., pes checkout a1b2c3).

What happens when committing in detached HEAD state:

- head_read() returns the commit hash stored directly in HEAD — this becomes the parent of the new commit.
- head_update() writes the new commit hash directly back into HEAD (since HEAD is not pointing to a branch ref file, there is no branch to advance).
- Each new commit extends the chain, but only HEAD tracks it. There is no branch pointer (no file in refs/heads/) that refers to any of these commits.
- As soon as the user switches to another branch, HEAD is overwritten with the new branch reference. The commits made in detached HEAD state become orphaned — no ref points to them.
- Orphaned commits are not immediately deleted; they remain in the object store until garbage collection runs. However, they are invisible to pes log (which only follows the current branch) and inaccessible through normal commands.

How a user can recover these commits:

- If the commit hashes are known (e.g., from terminal history or pes log output captured before switching): The user can run pes checkout <hash> to return to the last commit in the detached chain, then create a new branch at that point (pes checkout -b recovery-branch). This turns the orphaned chain into a reachable named branch.
- If the hashes are not known: Git provides git reflog, a local log of every HEAD movement. PES-VCS does not implement reflog in this lab, so recovery would require inspecting the object store directly — walking all objects, identifying commit objects that have no incoming references from any branch, and identifying the most recent ones by timestamp.
- Prevention: Users should always create a branch before making commits in detached HEAD state. The standard workflow is: pes checkout <hash>, then immediately pes checkout -b new-feature before making any changes.

Phase 6: Analysis — Garbage Collection and Space Reclamation

Q6.1 — Finding and Deleting Unreachable Objects

Question: *Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to. Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.*

Answer

Algorithm — Mark-and-Sweep Garbage Collection:

Phase 1: Mark (find all reachable objects)

- Start with the root set: read every file in .pes/refs/heads/ and .pes/refs/tags/ to collect all branch and tag tip commit hashes. Also include the commit that HEAD points to directly (in case of detached HEAD).
- Use a HashSet (e.g., implemented as a hash table or a balanced BST) to store all reachable hashes. A HashSet provides O(1) average-case insertion and lookup, which is critical when processing hundreds of thousands of objects.
- Perform a breadth-first or depth-first traversal from each root commit. For each commit hash encountered: read the commit object, mark the hash as reachable, enqueue its parent hash(es) if not already marked, read the commit's tree hash and recursively mark it.
- For each tree hash: read the tree object, mark it as reachable, for each entry that is a blob: mark the blob hash as reachable, for each entry that is a subtree: recursively mark it (enqueue if not already visited).

Phase 2: Sweep (delete unreachable objects)

- Walk the entire .pes/objects/ directory tree using opendir/readdir (or find) to enumerate every stored object hash.
- For each object on disk: reconstruct its full hash from its directory path (e.g., 2f/ + 8a3b... = 2f8a3b...). Check membership in the reachable HashSet. If not present, the object is unreachable — delete the file.

Data structure: A HashSet of 64-character hex strings (SHA-256 hashes). In practice, storing the hashes as 32-byte binary values reduces memory by half. For 100,000 commits, peak memory is roughly 32 bytes × estimated object count ≈ tens of megabytes, which is trivially manageable.

Estimated object count for 100,000 commits and 50 branches:

- Commit objects: at least 100,000 commits (one per commit in history, assuming a linear chain; more with merges).
- Tree objects: each commit references at least one root tree. With an average project directory depth of 3 and 20 files, each commit could reference 5–10 tree objects. A conservative estimate is 3–5 trees per commit: roughly 300,000–500,000 tree objects, though unchanged subtrees are shared across commits, so in practice the total is much lower.
- Blob objects: blobs are deduplicated, so the total blob count grows only as unique file versions are added. For an active project, this might be 500,000–2,000,000 blobs over 100,000 commits.
- Total objects to visit during the mark phase: roughly 1,000,000–3,000,000. For 50 branches pointing to different tips, the traversal must process all reachable commits from all 50 tips.

With significant shared history, the set of unique commit hashes is exactly 100,000 (or somewhat more with branch divergence).
- The sweep phase visits every file on disk — all objects regardless of reachability — which is the same total count.

Q6.2 — GC Race Condition with Concurrent Commits

Question: *Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?*

Answer

The race condition:

A commit operation proceeds in multiple steps: write blob objects, write tree objects, write the commit object, then update the branch ref. Between the moment a blob is written to disk and the moment the branch ref is updated to point to the new commit, the new objects exist in the object store but are not reachable from any branch. During this window, they look exactly like orphaned objects to a concurrently running GC.

Concrete race condition scenario:

- T=0: Commit thread writes blob a1b2c3... to the object store. The object exists on disk, but no ref points to it yet.
- T=1: GC thread begins its mark phase. It reads all branch refs and builds the reachable set based on the current state of HEAD and all branches. Since the branch ref has not been updated yet, a1b2c3... is not in the reachable set.
- T=2: GC thread sweeps — it finds a1b2c3... on disk, determines it is not reachable, and deletes it.
- T=3: Commit thread writes the tree object that references a1b2c3..., then writes the commit object that references the tree, then updates the branch ref to the new commit hash.
- T=4: The repository is now in a corrupt state. The new commit references a tree that references a blob that no longer exists. Any attempt to read or check out this commit will fail with a missing object error.

How Git's real GC avoids this:

- Grace period (the primary defence): Git's GC refuses to delete any object that was modified within the last 2 weeks (configurable via gc.pruneExpire). Newly written objects are always recent, so they survive GC even if they are not yet reachable. This makes the window of vulnerability effectively zero for normal workflows.
- Loose object timestamps: Since each object is a separate file, the filesystem mtime tells Git exactly when it was written. The GC checks mtime before deleting and skips anything recent, regardless of reachability.
- Lock files: Git uses .git/gc.pid lock files to prevent two GC processes from running simultaneously. However, this does not prevent GC from running concurrently with a commit — that is handled by the grace period.
- Ref advertisement: In networked contexts (e.g., git push), Git's pack-objects protocol advertises refs to the receiving side before sending objects, ensuring the receiver never treats just-received objects as unreachable during a concurrent GC.

- Design implication for PES-VCS: A safe implementation of pes gc should skip any object whose file mtime is less than a configurable threshold (e.g., 30 minutes) old. This provides protection against the race without requiring complex locking or atomic multi-step transactions.

Submission Checklist

Screenshots Required

Screenshot ID	Description
1A	Output of ./test_objects showing all tests passing
1B	find .pes/objects -type f showing sharded directory structure
2A	Output of ./test_tree showing all tests passing
2B	xxd of a raw tree object (first 20 lines)
3A	pes init → pes add → pes status sequence output
3B	cat .pes/index showing the text-format index
4A	pes log output with three commits
4B	find .pes -type f | sort showing object store growth
4C	cat .pes/refs/heads/main and cat .pes/HEAD
Final	Full integration test (make test-integration) output

Code Files Required

File	Implementation
object.c	Object store: object_write, object_read
tree.c	Tree serialization: tree_parse, tree_serialize, tree_from_index
index.c	Staging area: index_load, index_save, index_add, index_status
commit.c	Commit and history: head_read, head_update, commit_create
pes.c	CLI entry point: cmd_commit

Analysis Questions Answered

Question	Topic
Q5.1	Implementing pes checkout — files changed, working dir sync, complexity
Q5.2	Detecting dirty working directory conflicts using index and object store
Q5.3	Detached HEAD state — commits, orphaning, and recovery
Q6.1	GC mark-and-sweep algorithm and object count estimation
Q6.2	GC race condition with concurrent commits and Git's mitigations

Commit History Requirement

Minimum 5 detailed commits per phase with descriptive commit messages. More granular commits per phase are preferred and may improve your grade.